

This course has already ended.

« Chapter 19.1: Design patterns (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 19 (https://plus.cs.aalto.fi/studio_2/k2022/w19/) / Chapter 19.2: Own graphical user interfaces Chapter 19.3: Own graphical user interfaces, animation

Chapter 19.2: Own graphical user interfaces

About this page:

Key questions How can I draw own graphics in a window?

Topics Using graphics with Swing user interface.

What Will I Do? Ready-made examples, reading and answering to a few questions.

Indicative difficulty level: Easy

Rough Estimate of Workload:? 1 hour

Point Value: S10

Related Projects: GraphicsExamples
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/GraphicsExamples/)

Preface

If you no longer remember anything about the chapter 12.3 (<https://plus.cs.hut.fi/o1/2021/w12/ch03/>), which presented the basics of graphical user interfaces, it may be a good idea to start by revisiting the chapter.

The basic ideas can also be read in the first edition of the Programming in Scala book, (<http://www.artima.com/pins1ed/gui-programming.html>).

scala.swing

Scala uses Java libraries as its graphics library. They are working well for all major operating systems. Java has two (or currently up to three) built-in graphics libraries, the oldest of which is **AWT** (Abstract Window Toolkit). AWT's graphical components such as buttons are no longer used today - "newer" **Swing** has replaced it. However, AWT contains the most important classes for event listening and 2D graphics, so it is still active. The latest Java graphics library is **JavaFX2**, which we do not discuss in this course. On top of that there has been built the **ScalaFX** library. You can find the ScalaFX tutorials in the left hand menu.

Swing is the most widely used Java's default library, and using it in Scala has been supported by scala.swing library, which can be used closer to Scala's programming style. The library carries out the basic features, but remains loyal to Java's original library. In areas where you need some more special swing knowledge, you can almost always consult the corresponding Java swing material. This is often necessary.

Swing documentation and deployment

In Chapter 12.3 (<https://plus.cs.hut.fi/o1/2021/w12/ch03/>), a number of different scala.swing components were presented. Their scaladoc documentation is here (<http://www.scala-lang.org/api/2.12.0/scala-swing/scala/swing/index.html>).

In Scala version 2.11, some parts of the basic library were set aside from the core library, and swing was one of these. That's why you won't find it among the other Scala API documentation. To enable it, you have to tell IntelliJ (or whatever environment you are using) separately where the JAR package inside the scala.swing library is. The course project S2Library includes the swing package.

You can also add the Scala swing package to the project using Maven as shown in the Friday lectures.

Thus, if you try to use the examples in your own project, add to your project the dependency of S2Library project or Maven

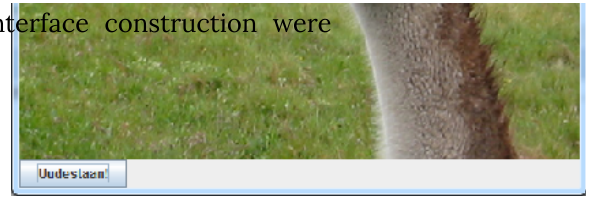
The basics of building a user interface

(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/llama.png)

If you have completely forgotten what was taught about user interfaces in the Programming 1 course, you should read Chapter 12.3 (<https://plus.cs.hut.fi/o1/2021/w12/ch03/>), execute the examples and decide on the fate of the lama.



In the Lama example, the basics of event listening and user interface construction were presented.



My own graphics in the user interface

Chapter 12.3 (<https://plus.cs.hut.fi/o1/2021/w12/ch03/>) used existing user interface components. If and when you want to develop graphical components with their own appearance, they are built on the existing (Scala) swing components. A typical base for a new component is the class `Panel`, which contains everything you need to directly connect it to the user interface, but with no extra functionality.

The code snippet below provides surreal art.

```
import scala.swing._
import java.awt.Color

object SimpleGraphics extends SimpleSwingApplication {

  def top = new MainFrame {
    title      = "Ball"
    contents = new BallPanel
    size       = new Dimension(400, 400)
  }

  class BallPanel extends Panel {

    override def paintComponent(g : Graphics2D) = {
      g.setColor(Color.blue)
      g.fillOval(20, 20, 300, 200)

      g.setColor(Color.black)
      g.drawString("Ceci n'est pas un ovale.", 115, 250)
    }
  }
}
```

Let us consider this piece by piece. The import statements are not very interesting except, perhaps, that they introduce the Java class `Color`. As noted earlier, Scala code can use Java in virtually the same way as if the libraries were written in Scala.

```
object SimpleGraphics extends SimpleSwingApplication {

  def top = new MainFrame {
    title      = "Ball"
    contents = new BallPanel
    size       = new Dimension(400, 400)
  }
}
```

`SimpleSwingApplication` is a subset that facilitates the creation and deployment of `scala.swing` applications. By inheriting the object from this class, a graphical application is obtained whose starting point is in this object.

The SimpleSwingApplication class is defined by the abstract method top, where the class's user typically defines the main window of his/her program. In this case, we use another scala.swing class, MainFrame, which creates the (main) application window. If MainFrame is closed, the entire application closes.

MainFrame includes several graphical areas: title, possible menus, etc. The area where the actual content of the window will be presented, can be manipulated through the contents variable. In our example, we place a BallPanel instance in it.

The size of the components can be directly or indirectly modified. In this size, size sets the component size directly. However, for most components that are intended to be part of the user interface, the values preferredSize, minimumSize, and maximumSize are used. This allows the user interface to change the size of the component, respecting these desires.

Method paintComponent

The usual Panel-class instance is drawn as a white background rectangle. If other components have been added to the panel, these components are drawn into the panel area. Instructions for drawing all components can be found in the method paintComponent. The BallPanel can be plotted differently from the default panel by overriding the operation of this method.

Method paintComponent (g: Graphics2D) gets a Graphics2D object that can be considered a simplified canvas (drawing area). The canvas has a wide range of methods for drawing different geometric shapes, lines, text, images, etc.

```
class BallPanel extends Panel {  
  
  override def paintComponent(g : Graphics2D) = {  
    g.setColor(Color.blue)  
    g.fillOval(20, 20, 300, 200)  
  
    g.setColor(Color.black)  
    g.drawString("Ceci n'est pas un ovale.", 115,250)  
  }  
}
```

In our example, we first change the color and then move the filled ellipse. Finally, the color is changed to black and an explanatory text is written under the ellipse.

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 6

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 3 / 3

Test by changing the program, or conclude which one of the following is true:

- ☐ Origo is in the lower left corner. The x-coordinate increases to the right, the y-coordinate increases upwards.

☒ **Origo is on the top left. The x-coordinate increases to the right, the y-coordinate increases downwards.**

✓ Correct!

Question 2 4 / 4

Test by changing the program, or conclude what happens if line **size = new Dimension (400, 400)** is deleted.

- ☐ The window fills the whole screen when it is started.
- ☒ **The window is created as uselessly small.**
- ☐ The window is created just like the size that the drawn graphics fit into it.
- ☐ The window is created to the size that the title can fit.

✓ Correct!

Question 3 3 / 3

If you set `g.`scale` (-1, -1)` and `g.translate(-400, -400)` as the first lines of ``paintComponent``.

- ☐ the image shrinks
- ☐ The image is mirrored from left to right
- ☒ **the image rotates the other way round.**

✓ Correct!

Submit

Who calls method `PaintComponent`?

The `paintComponent` method is called whenever a component is to be drawn on the screen. Drawing can be triggered either by resizing or moving the window, etc., which would anyway require the component to be drawn in a new size, etc. The drawing can also be started on request.

For any user interface component, a method `repaint` may be called. It gives the user interface a request that that component and all components contained therein should be redrawn when possible. When this moment emerges, the so-called event listener thread (which we will talk about after a while) performs the method `paint` in the component, which, among others, calls the `paintComponent` method that we just replaced. `Paint` also takes care of drawing the aforementioned children by recursively reclaiming "child components" `paint` methods.

This kind of structure is called the Composite design pattern, which we introduced in the previous chapter.

Bitmap graphics and image files

`Graphics2D` can be used to draw both vector graphics as above and bitmap graphics. Bitmaps are described with the help of class `scala.awt.Image`, although typically we use its subclass ``BufferedImage``.

`BufferedImage` objects can be downloaded from the file with the Java `ImageIO` library and drawn on the screen using the `drawImage` method of `Graphics2D`.

```
import javax.imageio.ImageIO
val example = ImageIO.read(new File("example.png"))

...

override def paintComponent(g: Graphics2D) = {

    // Draws the image at coordinates 0, 0
    // The last parameter is an ImageObserver which we won't use here

    g.drawImage(example, 0, 0, null)

}
```

Of course, `BufferedImage` can also be created from scratch, and we can draw into it by using the familiar `Graphics2D` class. Finally, the result can be saved as a file using the `ImageIO` library.

```
import javax.imageio.ImageIO
import java.io.File
import java.awt.image.BufferedImage
import java.awt.{Graphics2D, Color}

object Images extends App {

    val pic = new BufferedImage(300, 300, BufferedImage.TYPE_INT_ARGB)
    val g = pic.getGraphics.asInstanceOf[Graphics2D]

    g.setColor(Color.blue)
    g.fillOval(10,10,290,290)

    ImageIO.write(pic, "png", new File("artwork.png"))
}
```

In addition to the size of the image, the `BufferedImage` constructor needs to know what kind of pixels there are in our picture. `ARGB` = Alpha (transparency), Red, Green and Blue. Typically, the choice is between whether there is the transparency channel `A` or mere colors. The order of colors in a pixel is usually `RGB` and changes to it are only relevant in some hardware-related things.

The method `getGraphics` gives a "canvas" that allows you to modify the bitmap image. The methods are exactly the same as if the drawing were made for the user interface component.

The return value of the `getGraphics` method is backward compatible with the very first Swing versions. Although, according to the documentation, the return value of the `getGraphics` method is `Graphics`, it is practically always the subtype of `Graphics2D`. By giving this type information to the Scala compiler with the `asInstanceOf` method, we can enable the use of the newer `Graphics2D` class.

The graphics we draw now ends up in `BufferedImage`.

...and then to the disc as a `png` image. In addition, the storing method requires information about the format used and the file where the result is written.

Examples

The demo projects of the chapter includes a few examples of ready-made examples of graphics, event listening and animation that can be executed, sprites and bitmap graphics:

GraphicsExamples
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/GraphicsExamples/)



Where can I get more information?

The Scala.swing components' interface differs partly from Java, but from the point of view of the basics and possibilities, Java swing-demos are worth looking through: Using Swing Components (<http://docs.oracle.com/javase/tutorial/uiswing/components/index.html>)

Find more information about 2D graphics features in the same place: 2D-Graphics (<http://docs.oracle.com/javase/tutorial/2d/index.html>)

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

60

Show anyway

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

Submit an update

« Chapter 19.1: Design patterns (<https://plus.cs.aalt...>)

Chapter 19.3: Own graphical user interfaces, anima...